

Certified but Compromised: Breaking and Fixing Lightweight Proofs of Inference

Final Report — Trustworthy Machine Learning, Spring 2026

Ofek Lavi
Tel Aviv University
Tel Aviv, Israel
ofeklavi@mail.tau.ac.il

Erel Brazilay
Tel Aviv University
Tel Aviv, Israel
erelbrazilay@mail.tau.ac.il

Edo Kadosh
Tel Aviv University
Tel Aviv, Israel
edo@mail.tau.ac.il

Abstract

When a large model runs in the cloud, its users cannot tell whether an answer really came from the agreed weights. zkSNARK proofs of inference solve this but take minutes per query, so Anchuri et al. (SaTML 2026) proposed a lightweight alternative: the server commits to the whole computation and the client re-checks one random path through the network. We reimplement this protocol and break it by changing a single neuron: the verifier still accepts the tampered answer 99.8% of the time, and a backdoor built this way costs nothing in clean accuracy. We show that choosing more cleverly which neurons to check cannot fix this. We then design a new defence that checks *every* layer at once. It verifies one random linear combination per layer (Freivalds’ check) against a Reed–Solomon/Merkle commitment to the weights, in exact integer arithmetic. It catches any wrong value with probability $1 - 2^{-\lambda}$ and needs no trusted setup. On models from LeNet-5 to Llama-2-13B it rejected all 2,028 attacks and accepted all 3,174 honest queries. Its prover is one to four orders of magnitude faster than published zkSNARK provers on the same models. The price is a larger proof and a slower verifier.

Keywords

verifiable inference, proof of inference, integrity attacks, Freivalds’ algorithm, zkML

1 Introduction

Suppose a company pays a cloud provider to answer its customers with a specific large language model, for example a model it has licensed or fine-tuned. Every answer that comes back is just text. Nothing in it shows that the provider really ran that model with the agreed weights. It could have used a cheaper or more heavily quantised copy, a model with a few weights edited, or a model that behaves normally except on certain inputs. The customer cannot simply re-run the model to check: not having to run it is the whole reason to use the cloud. A *proof of inference* closes this gap. With every answer the provider also sends a piece of evidence, and the client runs a cheap check that passes only if the answer came from the agreed weights on this input.

The strongest proofs of inference are zkSNARKs [6, 12, 21, 25, 27]. Their proofs are tiny and quick to check, but producing one takes far longer than the inference itself. zkLLM, for example, needs about ten minutes on an A100 GPU for one Llama-2-7B prompt [27]. Anchuri et al. [2] recently proposed a much lighter approach. The provider commits to the full execution trace (every

activation) with a Merkle tree. The client then checks the computation along *one random path* from an output neuron back to the input, against a commitment to the weights. Proving drops from minutes to milliseconds. The security argument is statistical: a different model produces a trace that differs from the correct one in many places, so a random path is very likely to hit one of them.

What we did. Our intermediate report started from one observation. Such a scheme certifies that the served model is *functionally close* to the committed one, not that it *is* the committed one, and backdoored models [15] are the natural counterexample. We reimplemented the protocol from scratch (no code was released) and reproduced its main claims. Then we asked what happens if the adversary does not swap the model but changes only what it needs to. The answer is simple and severe. A prover can run the right model, overwrite *a single neuron*, and honestly recompute everything after it. The resulting trace is wrong in exactly one place. A random path passes through that place with probability $1/N$ in a layer of width N , so on our MNIST network the verifier accepts the tampered answer 99.8% of the time. Triggering the change only on chosen inputs gives a backdoor with no accuracy cost at all on clean inputs. The paper notes that one path cannot detect a single changed node with probability above $1/N$. It argues that this does not appear to be a practical limitation for the substitute-model adversary it targets, and suggests sampling more paths, or sampling adaptively, as mitigations. We show that the attack is practical, and that neither mitigation can repair it (Figures 1a and 1b).

This led us to a different design (Figure 1c). Instead of checking a few neurons exactly, we check *every* layer at once with one random linear combination of all its neurons (Freivalds’ algorithm [11]). When the client does not hold the weights, it checks this combination against a Reed–Solomon-encoded, Merkle-committed copy of them, in the style of Ligerio [1]. The client therefore still stores only 32 bytes per weight matrix. To make the check exact, the model runs as an integer (int8) network that gives bit-for-bit the same result on the provider’s GPU and the client’s CPU. The prover’s work stays close to plain inference. The chance of catching a wrong value, anywhere in the network, rises from $1/N$ to $1 - 2^{-\lambda}$.

Contributions.

- **An attack** on the protocol of [2]. Changing one neuron changes the answer. The verifier accepts the result with probability $1 - 1/N$ (99.6–99.8% measured), including a trigger backdoor with zero clean-accuracy gap and 100% attack success.
- **An impossibility argument** for sampling-based fixes. Uniform sampling is already the best rule that ignores the

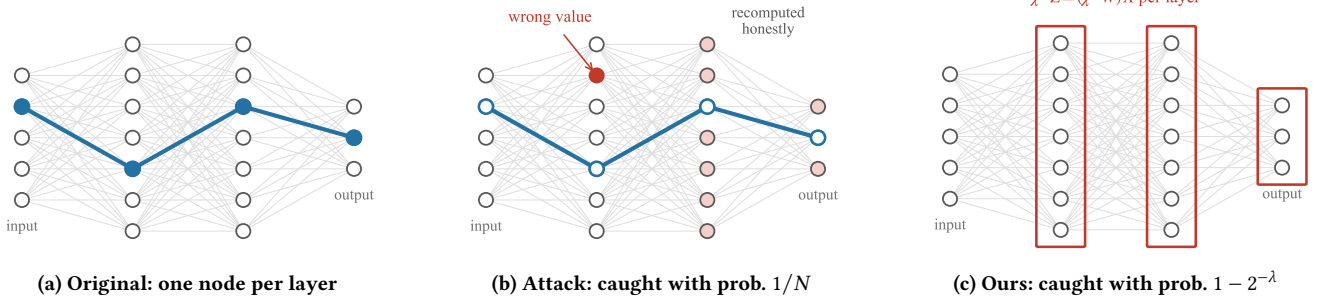


Figure 1: Checking one path versus checking every layer.

claimed values. The natural rule that looks at them has a blind spot, and our attacker reaches exactly zero detection through it. Reaching 2^{-40} forces the original protocol to open essentially the whole model and trace.

- **A new defence** with $2^{-\lambda}$ statistical soundness and no trusted setup. It covers whole CNNs (convolutions, pooling, residual blocks) and transformers (normalisation, GELU/SiLU, RoPE, softmax attention), with the weights either committed or known to the client.
- **A like-for-like evaluation** on the models used by the original paper and by 17 other systems: 6 trained CNNs and 7 LLM architectures up to 13B parameters. Every attack was rejected, and the prover is one to four orders of magnitude faster than zkSNARKs. All code and raw measurements are released with this report.

2 Related Work

zkSNARK proofs of inference. A long line of work compiles neural networks into succinct zero-knowledge proofs. For CNNs these include sum-check/GKR provers [14] such as zkCNN [21] and zkPyTorch [32], and pairing- or Plonk-based systems such as vCNN [19], ZEN [8], ZENO [9], ZKML [6, 17], EZKL [35] and Bionetta [34]. Mystique [31] is an interactive, VOLE-based alternative. For LLMs they include zkLLM [27], zkGPT [25], Deep-Prove [12], ZKTorch [5], Jolt Atlas [4] and NanoZK [30]. These systems give computational soundness and small proofs, but their provers are far slower than inference: from half a second for LeNet-5 to minutes or hours per query for larger models.

Checks when the verifier knows the weights. SafetyNets [13] uses interactive proofs for networks with quadratic activations. Slalom [29] checks every linear layer of a GPU computation from inside an SGX enclave with Freivalds’ algorithm, optionally with a precomputed secret challenge. Maverick [22] brings sparse Freivalds checks and a linear code to LLM inference, for a client that preprocesses the weights. LAMP [18] combines Freivalds’ check with proximity testing to prove a single committed matrix product.

Sampling, fingerprints and optimistic protocols. Anchuri et al. [2] replace proofs with spot checks along random paths and argue security through trace separation. SLP [20] samples which layers receive a SNARK. TOPLOC [24] and SVIP [28] fingerprint

hidden states and give empirical guarantees. Verde [3], opML [7] and TAO [33] resolve disputes between parties, and need at least one honest party or a challenger.

Where we fit. Our defence reuses two known building blocks: Freivalds’ check (as in Slalom and Maverick) and a linear-code commitment to the weights (as in Ligero and LAMP). We combine them into an end-to-end protocol for whole CNN and transformer inference, in the committed-weights setting of [2]. What is new here is (i) the attack and the analysis showing why the sampling approach cannot reach full soundness, (ii) the complete protocol: exact integer graphs, non-weight operations recomputed by the verifier, and parameters derived from λ , and (iii) a like-for-like comparison on the same models as the original paper and the zkSNARK literature.

3 Technical Approach

3.1 Setting and threat model

A model M has a public architecture and secret or public weights. Before any query, a commitment C_M to the weights is computed correctly and given to the client, for example by the model owner. As in [2], we assume this step is honest. The client sends a query x . The provider returns a claimed output y and a proof. The provider may deviate arbitrarily. We want *completeness*: honest answers are always accepted. We also want *full soundness*: any $y \neq M(x)$ is rejected except with a small probability ϵ . The original paper proves a weaker notion, *other-model soundness*, in which the adversary must honestly run some different model. We consider three trust settings. In **C** the client holds only C_M (32 bytes per weight matrix), as in [2] and the zkSNARKs. In **K** the client knows the weights. **Kpre** is **K** with a secret, precomputed challenge, as in Slalom and Maverick.

3.2 The original protocol and our attack

In the protocol of [2] (“RandPathTest”), the prover commits to every activation a_j of the forward pass with a Merkle tree. The verifier then picks a random output neuron and walks back to the input, picking a random parent at each layer. At every node j on the path it opens the parents’ activations and the weights w_{ij} from C_M , and checks the local relation $a_j = \phi(\sum_{i \in G_j} w_{ij} a_i)$. One path costs one weight row per layer, which is why proving is so cheap.

The single-neuron attack. Our adversary runs the correct model on x and picks one neuron j in some layer. It replaces a_j

by a value chosen so that the output becomes the target (a wrong class, or any class the attacker wants). It then recomputes all later layers *with the true weights*. Every node above j is consistent with its parents, and every node below j is untouched, so exactly one node in the whole trace is wrong. The verifier notices only if its path visits j . This happens with probability $1/N$ in a layer of width N , or $1 - (1 - 1/N)^k$ with k paths. Applied only to inputs that contain a trigger, the attack becomes a backdoor. Clean answers are bit-for-bit the honest ones, so the clean-accuracy gap is exactly zero.

Why sampling more cleverly cannot fix it. A natural fix is to sample “where it matters” instead of uniformly. We proved two statements that rule this out.

THEOREM 3.1 (UNIFORM SAMPLING IS MINIMAX-OPTIMAL). *Consider any sampler whose choice of nodes does not depend on the claimed trace and that opens at most k nodes of a layer. Let U be the neurons of that layer whose change can flip the output, and $q(v)$ the probability that node v is opened. An adversary who knows the sampler tampers with the least-visited node in U , so detection is at most $\min_{v \in U} q(v) \leq k/|U|$. Equality holds only for uniform sampling.*

The bound follows because the minimum is at most the average, and the $q(v)$ sum to at most k . Constant detection therefore needs $k = \Omega(N)$ openings per layer, which is linear in the width: exactly the cost the protocol was built to avoid. Samplers that *do* look at the claimed values fare no better in the worst case. The most effective one we found against the naive attack steps to parent i with probability proportional to its contribution $|w_{ij}a_i|$. It gives weight zero to any neuron whose claimed activation is zero. The adversary therefore flips the output by setting a few neurons to exactly zero, and its detection is exactly 0. Mixing in a uniform floor restores some detection (about $10\times$ uniform on average in our experiments), but it stays at a few percent.

3.3 Our defence: checking every layer at once

The idea fits in one sentence. Instead of checking a few neurons exactly, the verifier checks *one random combination of all the neurons of a layer*; a wrong value anywhere in the layer changes that combination, except with negligible probability. We now build this up in four steps and then give the complete protocol.

Step 1: an exact integer model. A random combination is only a reliable test if both sides compute exactly the same numbers, so the model runs in integers. Following standard integer-only inference [16], each weight layer computes $Z = WX + b$, where W and X are int8 with per-channel scales, BatchNorm is folded into W and b , and all sums fit in 32 bits. The next layer’s input is obtained by fixed-point requantisation followed by the usual activation or pooling. Convolutions become matrix products through im2col. The prover’s GPU computes these products in float32 chunks of at most 1,024 terms; every partial sum stays within 2^{24} and is therefore exact, so the GPU result is bit-identical to the client’s CPU. This also removes the floating-point tolerance that the original protocol needs, which in our reproduction was mandatory and gave the attacker a free budget.

Step 2: only weight products need a proof. For every weight layer ℓ the prover sends the claimed pre-activations Z_ℓ (the *claims*). Everything that does not involve the weights, the verifier computes itself from values it has already checked: requantisation, ReLU, pooling, residual additions, LayerNorm/RMSNorm, GELU/SiLU (by table lookup), RoPE, softmax, and the attention products $QK^\top$ and AV . So the input X_ℓ of every weight layer is always the verifier’s own, and the only thing left to check is $Z_\ell = W_\ell X_\ell + b_\ell$. Because the verifier builds each X_ℓ from the previous layer’s checked claims, the layers are chained: a prover cannot answer one layer consistently and feed a different value into the next.

Step 3: Freivalds’ check. Write $A_\ell = [W_\ell \mid b_\ell]$, a matrix with N rows (one per output neuron) and row length k , and let M be the number of columns of X_ℓ (pixels or tokens). The verifier draws a random vector $\chi \in \mathbb{F}_p^N$, one entry per output neuron, and checks

$$\chi^\top Z_\ell = \underbrace{(\chi^\top A_\ell)}_{u_\ell} [X_\ell; \mathbf{1}]. \quad (1)$$

Why does this catch every error? Let $D = Z_\ell - A_\ell [X_\ell; \mathbf{1}]$ be the difference between the claims and the truth. The two sides of (1) differ by $\chi^\top D$. If D has a non-zero entry in row i and column m , then entry m of $\chi^\top D$ equals $\chi_i D_{im}$ plus terms that do not involve χ_i . For any values of the other entries, exactly one value of χ_i makes it zero, so a wrong layer passes with probability at most $1/p$. With r independent vectors this drops to p^{-r} . It does not matter how many values are wrong or where they are: one neuron or all of them is caught equally well. The check is also cheap: both sides cost $O((N+k)M)$, against $O(NkM)$ for recomputing the layer.

We work in the BabyBear field $\mathbb{F}_p$ with $p = 15 \cdot 2^{27} + 1 \approx 2^{31}$, which makes the argument exact (no rounding, no overflow). The verifier range-checks every claim ($|z| < 2^{29}$), so two different integer claims are always two different field elements.

Step 4: committed weights. In setting C the verifier does not have A_ℓ , so it cannot compute $u_\ell = \chi^\top A_\ell$; the prover sends it. The remaining question is how the verifier can trust u_ℓ while holding only a 32-byte commitment. Checking a few entries of u_ℓ against the weights would be weak: a prover who changes one of the k entries escapes a check of t random entries with probability $1 - t/k$. We therefore commit to an *encoded* copy of the weights, in the style of Ligerio [1]. When the model is published, every row of A_ℓ is Reed–Solomon encoded [26] from length k to $n = 4k$, giving a matrix $\hat{A}_\ell$. The commitment C_M is a Merkle root [23] over the n columns of $\hat{A}_\ell$, 32 bytes per weight matrix. Encoding is linear, so $\chi^\top \hat{A}_\ell = \text{Enc}(\chi^\top A_\ell)$: the combination of the encoded rows is the encoding of the true u_ℓ . The verifier encodes the claimed u_ℓ itself, picks t random column positions j , and checks $\chi^\top \hat{A}_{\ell,j} = \text{Enc}(u_{\ell,j})$ on the opened columns. If u_ℓ is wrong, $\text{Enc}(u_\ell)$ and $\text{Enc}(\chi^\top A_\ell)$ are two different Reed–Solomon codewords, which agree on at most $k-1$ of the n positions. The code has spread any change over at least three quarters of the positions, so each opened column catches a wrong u_ℓ with probability at least $3/4$, and t columns leave at most $((k-1)/n)^t \approx 4^{-t}$.

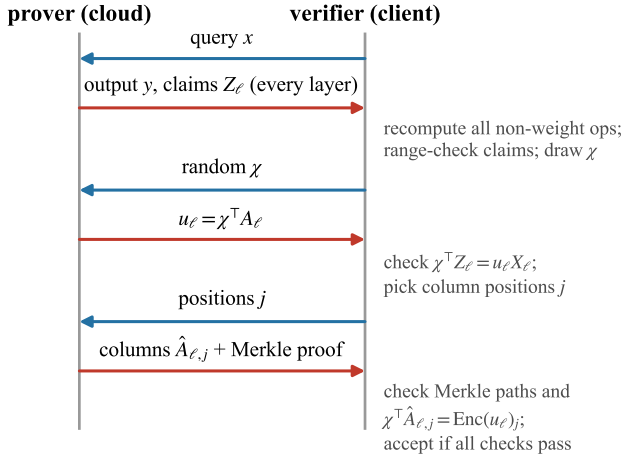


Figure 2: One query of our protocol (committed weights).

The complete protocol. Setup happens once per model: the owner encodes and commits the weights and publishes C_M . Then, for each query (Figure 2):

- (1) The client sends x .
- (2) The prover runs the integer model and returns the output y and the claims Z_ℓ of every weight layer.
- (3) The verifier range-checks the claims, recomputes every non-weight operation (obtaining each X_ℓ and checking that it leads to y), and draws r random vectors χ per layer.
- (4) The prover returns $u_\ell = \chi^\top A_\ell$ for every layer.
- (5) The verifier checks (1) for every layer and draws t random column positions.
- (6) The prover opens those columns of every $\hat{A}_\ell$ with one Merkle multiproof per matrix; the verifier checks the Merkle paths against C_M and the column equations.

The verifier accepts only if every check passes. The order matters: the claims are fixed before χ is drawn, and u_ℓ before the columns are chosen, so the prover can never adapt a wrong value to the randomness that will test it.

THEOREM 3.2 (SOUNDNESS). *If C_M is honest, then for any prover and any claimed $y \neq M(x)$, the verifier accepts with probability at most $\varepsilon = \sum_\ell [p^{-r} + (\frac{k_\ell-1}{n_\ell})^t]$.*

Sketch. If $y \neq M(x)$, some claim is wrong; let ℓ be the first layer with a wrong claim. Its input X_ℓ is correct, because the verifier computed it from correct claims. If u_ℓ is correct, (1) fails except with probability p^{-r} (Step 3). If u_ℓ is wrong, a column check fails except with probability $((k_\ell - 1)/n_\ell)^t$ (Step 4). A union bound over the layers finishes the proof. $\square$

The guarantee is statistical: it needs no assumption about how models behave (no trace separation), no trusted setup, and only a collision-resistant hash for the Merkle tree. For a target λ we set $b = \lambda + \log_2(2L)$ for L weight layers, and use $r = \lceil b/\log_2 p \rceil$ repetitions and $t = \lceil b/\log_2(n/(k-1)) \rceil$ columns, which makes $\varepsilon \leq 2^{-\lambda}$. For VGG-16 this means $r=5, t=67$ at $\lambda=128$, and $r=2,$

$t=23$ at $\lambda=40$. The protocol can also be made non-interactive by deriving the challenges from the transcript with SHAKE-256 (Fiat-Shamir [10]), with 64 extra bits against a prover who retries many times. When the client knows the weights (setting K) it computes u_ℓ itself and steps 4 and 6 disappear. In Kpre the secret χ and u_ℓ are computed once per model, so the proof is just the claims.

Costs. The prover runs the forward pass, computes u_ℓ and opens t columns. It does not store the encoded weights, and recomputes each opened column from W_ℓ , so its extra work is $O((r+t) \sum_\ell N_\ell k_\ell)$, independent of the number of pixels or tokens. The verifier does $O(r \sum_\ell (N_\ell + k_\ell) M_\ell)$ work plus the non-weight operations. The proof is dominated by the claims, $4 \sum_\ell N_\ell M_\ell$ bytes, the size of the pre-activations. It adds $4r \sum_\ell k_\ell$ bytes for the u_ℓ , $4t \sum_\ell N_\ell$ bytes for the columns, and the Merkle hashes. Unlike a SNARK proof, it therefore grows with the activations (image size or prompt length) rather than staying constant.

3.4 Implementation

We wrote everything in Python and PyTorch: the original protocol, the attacks, the analysis and the new defence. The prover runs on a GPU and the verifier on a CPU. Field arithmetic uses exact limb decomposition, Reed-Solomon encoding uses an NTT, and Merkle trees use SHA-256 with multiproofs. The original protocol is also re-implemented on the same integer graphs as a like-for-like baseline. Its exact visit probabilities are computed by dynamic programming (checked against Monte Carlo), and openings are shared between paths. The code has 209 tests, including the forgeries of Section 4.3.

4 Results

4.1 Experimental setup

Models. We use the models that prior work reports on. LeNet-5 (MNIST) and VGG-11, VGG-16 and ResNet-18 (CIFAR-10) are the CNNs of zkCNN, ZKML, ZENO, Bionetta and EZKL. A 224×224 ResNet-18 dog-vs-cat classifier matches the model of [2], and a dog-vs-squirrel ResNet-18 plays the role of their substitute model $\tilde{M}$. The MNIST MLP (784–512–256–10) is the network of our attack experiments. We trained all of them ourselves and quantised them to int8. The int8 accuracy (Table 2) is within 0.4 points of the float model for every CNN. For language models we use the exact shapes of GPT-2, OPT-125M/1.3B/6.7B, Llama-2-7B/13B and Qwen3-4B with random int8 weights, since the costs depend only on the shapes. Models with more than 12 blocks are built with 1 and 2 blocks and extrapolated linearly (all blocks are identical).

Hardware and baselines. The prover is one NVIDIA RTX 2080 Ti. The verifier is 8 threads of an Intel Xeon Silver 4114 on the university’s SLURM cluster. We compare with (i) the original protocol run on the same graphs and machine, and (ii) the published results of 17 systems, taken as reported with their own hardware and each checked against the source paper. Every run is stored as raw records (59,547 in total), from which all tables and figures are generated.

Table 1: Attacks on the original protocol (MNIST MLP, one path).

Attack	wrong nodes	accepted
Substitute model $\tilde{M}$	$\approx 570 / 778$	0.000
Inverse transform	$\approx 590 / 778$	0.000
Logit swap	256 / 778	0.000
Gradient reconstruction	778 / 778	0.000
Single neuron, layer 1 (ours)	1 / 778	0.998
Single neuron, layer 2 (ours)	1 / 778	0.996
Backdoor, triggered inputs	1 / 778	0.996
Stealthy (in natural range)	$\approx 6 / 778$	0.989
Zeroing vs. smart sampler	13 / 778	1.000

4.2 Breaking the original protocol

Reproduction. Our implementation behaves as the paper describes. Over four substitute-model settings from the paper, honest provers were accepted in 100% of 3,000 runs each, and substitute models were caught 99.8–100% of the time, even an 8-bit quantised copy that agrees with M on every test prediction. We also found that the idealised exact-equality check accepts an honest prover only 17–32% of the time, because a whole-layer matrix product and a single-neuron dot product round differently. The tolerance of 10^{-4} that the paper prescribes is therefore mandatory, not optional.

The attack works. Table 1 lists, for each attack, how many of the 778 trace nodes are wrong and the probability that the verifier accepts (computed exactly, averaged over 200 queries). The paper’s own attacks try to make many nodes *almost* right, and are always caught. Changing a single neuron is accepted 99.6–99.8% of the time, exactly the $1 - 1/N$ we predict for layers of 256 and 512 neurons. As a backdoor (a 3×3 bright patch as trigger), it serves the honest output on 100% of clean queries, succeeds on 100% of triggered queries, and is accepted on 99.6% of them (the exact prediction is 99.8%). A stealthy variant keeps every forged value inside that neuron’s natural range (up to its 95th percentile), so a range check would not flag it. It needs only about 3–6 changed neurons and is still accepted about 99% of the time. More paths do not help much either: with 250 paths, a 250-fold larger proof, the attack is still accepted 61% of the time.

Smarter sampling does not help. Measured against an adversary who knows the rule, uniform sampling detects 0.20% of attacks. Static importance weighting drops to 0.11%, and gradient-saliency weighting, which a verifier cannot even compute without the full model, to 0.006%. Contribution weighting detects nothing: zeroing 13 of 512 neurons flips the prediction, and the real protocol accepted 2,000 of 2,000 runs (last row of Table 1). Zero is also the most common natural activation (65% of inputs for these neurons), so no statistical check flags it.

The same holds on larger models. On every CNN, changing one penultimate neuron flips 100% of predictions. One path catches it with probability 1/84 (LeNet-5) to 1/512 (VGG, ResNet), and the least-visited neuron is caught with probability as low as 1/28 million (orange bars in Figure 3a). Reaching 2^{-40} takes 2,316–14,182 paths, which opens every committed row: the proof becomes the

whole model plus the trace. The last three columns of Table 2 give these paths and bytes, next to our proof at $\lambda=40$. On Llama-2-7B one path detects 1/11,008 of single-neuron attacks, and 2^{-40} needs about 305,000 paths, i.e. the entire model (≈ 13.7 GB).

4.3 Our defence catches every attack

Across every model, mode and security level, our verifier accepted all 3,174 honest queries. It rejected all 1,854 attacks on the CNNs, all run at the weakest setting ($\lambda=40$, committed weights). These were: one changed value, one changed penultimate neuron, one changed logit, 1% of the weights perturbed, the substitute model $\tilde{M}$, and two forgeries built so that only one specific check can stop them. It also rejected all 174 attacks on the language models, from GPT-2 to Llama-2-13B, each a single pre-activation changed by +1 in a middle layer, in every mode. (Models above 12 blocks were attacked on their 1- and 2-block builds.) The check that fired was Freivalds’ check in 1,916 cases. The Reed–Solomon column check caught all 56 forged u_ℓ vectors, and the Merkle check all 56 forged columns, so every part of the protocol is needed and works (red bars in Figure 3a).

Figure 3b shows the trade-off between security and bytes sent. The dotted curves are the original protocol with 1, 2, ... paths: they rise slowly and reach 2^{-40} only when nearly the whole model is opened (the vertical lines). Our solid lines, at $\lambda = 40, 80, 128$, reach 48–58 bits with 67 kB–10.7 MB on the CNNs, and 150–153 bits with 131 kB–12.1 MB.

4.4 Cost

CNNs. Table 2 gives the int8 accuracy of each CNN and our cost with committed weights (C) and with known weights and a pre-computed challenge (Kpre). With committed weights our prover needs 11–105 ms per query, the verifier 21–370 ms, and the proof is 131 kB–12.1 MB. That prover time is 4–7× one forward pass of the same integer model, and with known weights the prover is the forward pass itself. For the VGGs and the CIFAR ResNet the proof is 2.4–4.4× smaller than the int8 model. For the 224-pixel ResNet it is about the size of the model (12.1 vs. 11.2 MB), because the activations of a large image are big.

Language models. Table 3 shows the same picture at LLM scale, for the next-token logits of one prompt (models above 12 blocks are extrapolated from 1- and 2-block builds). GPT-2 with a 64-token prompt is proved in 0.44 s and verified in 0.99 s with a 62 MB proof. Llama-2-7B takes 11.5 s and 7.2 s with a 762 MB proof, about 9× smaller than its int8 weights. At 2,048 tokens Llama-2-7B takes 32 s to prove, but the proof grows to 11.6 GB and verification to 3.7 minutes, since the verifier recomputes attention over all token pairs. With known weights (Kpre) the proof shrinks to the claims alone, for example 21.8 MB for GPT-2 and 349 MB for Llama-2-7B at 64 tokens.

Against zkSNARKs. Figure 4 places all systems on one chart: our runs in colour (hollow squares are 2,048-token prompts) and the published systems in grey, as reported on their own hardware. The orange cross is one path of [2]: cheap, but it catches a single-neuron attack only with probability 1/11,008. Table 4 gives exact ratios on matched models. In parentheses is the prompt length of our run, which equals theirs except for zkGPT (not stated) and

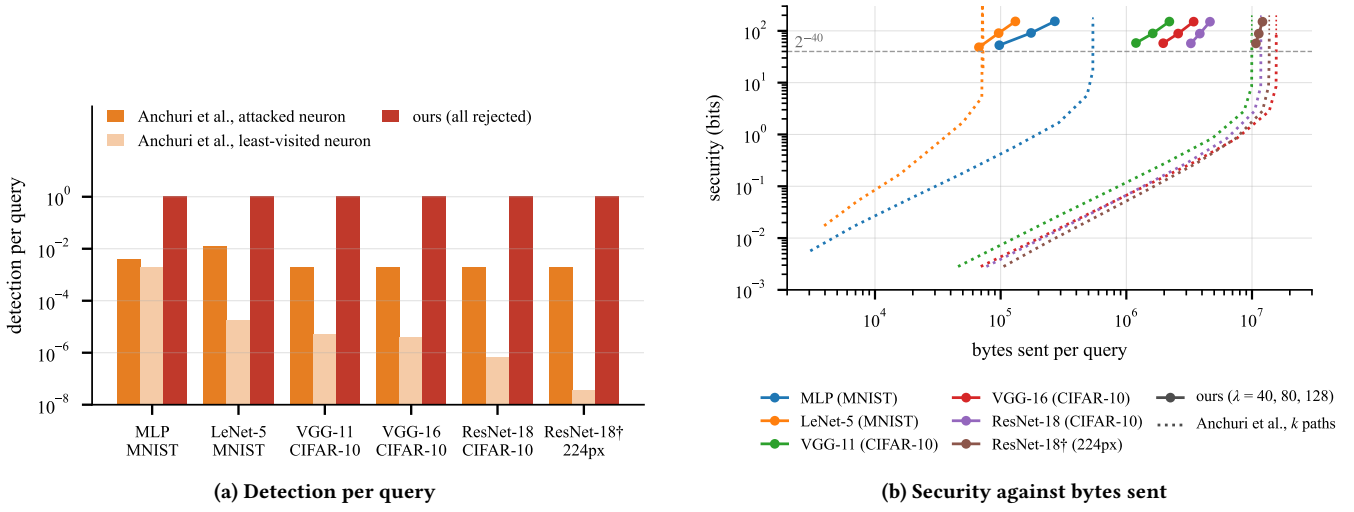


Figure 3: Catching the single-neuron attack.

Table 2: CNNs: one query at $\lambda=128$, and the cost of reaching 2^{-40} .

Model	Params	int8 acc.	ours, committed (C)			ours, Kpre		reaching 2^{-40}		
			prove	verify	proof	verify	proof	paths	bytes	ours
MLP (MNIST)	536K	97.7%	10.9 ms	21.4 ms	270 kB	3.7 ms	3.1 kB	7,084	542 kB	97.5 kB
LeNet-5 (MNIST)	62K	99.3%	16.3 ms	28.8 ms	131 kB	8.4 ms	26.1 kB	2,316	72.4 kB	67.2 kB
VGG-11 (CIFAR-10)	9.8M	91.1%	64.8 ms	134 ms	2.2 MB	29.7 ms	610 kB	14,182	10.0 MB	1.2 MB
VGG-16 (CIFAR-10)	15.2M	93.0%	90.7 ms	194 ms	3.4 MB	52.0 ms	1.1 MB	14,182	15.6 MB	2.0 MB
ResNet-18 (CIFAR-10)	11.2M	94.2%	69.1 ms	180 ms	4.6 MB	75.2 ms	2.5 MB	14,182	11.8 MB	3.3 MB
ResNet-18 [†] (224px)	11.2M	95.0%	105 ms	370 ms	12.1 MB	224 ms	9.9 MB	14,182	13.7 MB	10.7 MB

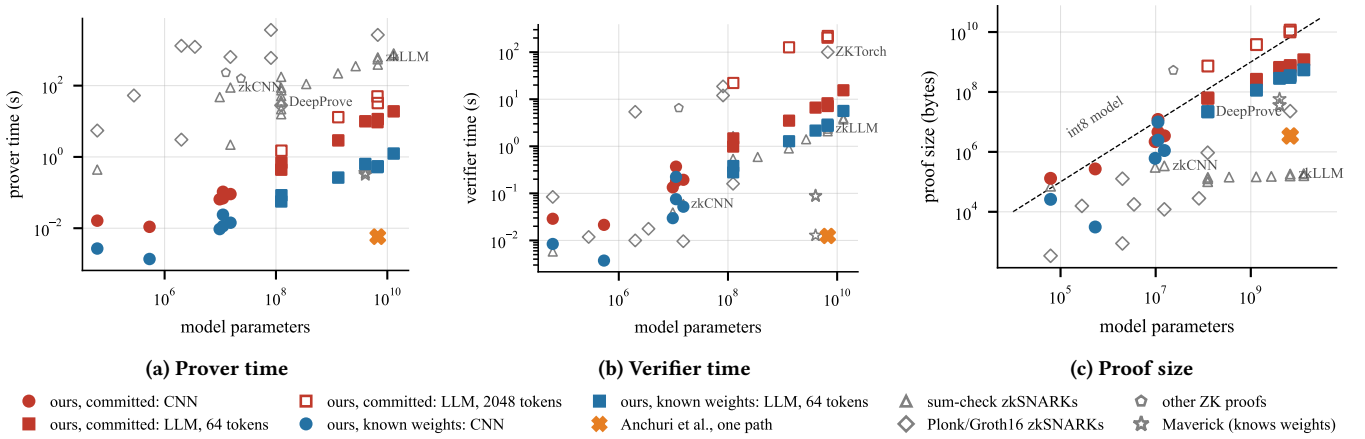
[†]The dog-vs-cat classifier of [2].

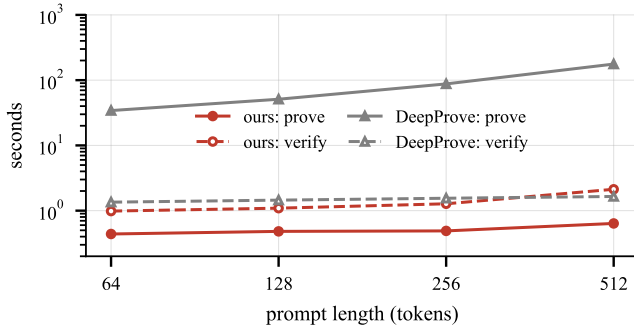
Figure 4: Cost against model size.

ZKTorch (1 token). Our prover is 27–7,024× faster than the CNN provers (zkCNN, Bionetta, ZKML), and 11–277× faster on language

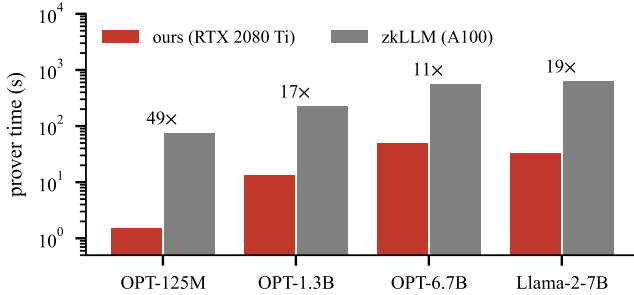
models (zkLLM, zkGPT, DeepProve, ZKTorch). Several of these systems ran on CPUs, so part of their gap comes from our GPU. zkLLM, however, ran on an A100, a stronger GPU than ours, and our prover is still 11–49× faster at the same 2,048-token prompts

Table 3: Language models: one prompt at $\lambda=128$.

Model	Tokens	committed weights (C)			Kpre	
		prove	verify	proof	verify	proof
GPT-2	64	439 ms	985 ms	62.1 MB	279 ms	21.8 MB
GPT-2	512	637 ms	2.1 s	213 MB	1.4 s	173 MB
OPT-125M	2,048	1.5 s	22.1 s	733 MB	21.3 s	692 MB
OPT-1.3B	2,048	13.0 s	2.1 min	3.8 GB	2.1 min	3.7 GB
Qwen3-4B	64	10.0 s	6.6 s	659 MB	2.2 s	284 MB
OPT-6.7B	2,048	49.9 s	3.4 min	10.1 GB	3.3 min	9.7 GB
Llama-2-7B	64	11.5 s	7.2 s	762 MB	2.7 s	349 MB
Llama-2-7B	2,048	32.4 s	3.7 min	11.6 GB	3.5 min	11.2 GB
Llama-2-13B	64	19.0 s	15.5 s	1.2 GB	5.6 s	547 MB



(a) GPT-2 against DeepProve



(b) Prover time at 2,048 tokens against zkLLM

Figure 5: Language models against zkSNARKs.

(Figure 5b). Against DeepProve on GPT-2 the gap grows with the prompt length (Figure 5a). The price is on the other side. Our verifier is usually slower: 2.8–5 $\times$ slower than the sum-check verifiers on small models, 20 $\times$ slower than ZKML, and 65–141 $\times$ slower than zkLLM at 2,048 tokens. It is faster than DeepProve up to 256 tokens and faster than ZKTorch. Our proofs are also larger, from 1.8 $\times$ (zkCNN, LeNet-5) up to 64,000 $\times$ (against zkLLM’s 141–183 kB proofs at 2,048 tokens).

Against Maverick. When the client knows the weights, our proof for Qwen3-4B with an 8-token prompt is exactly the size of Maverick’s (36.1 MB) [22]. Here we use Maverick’s setting: $\lambda=40$ and one client thread. Maverick’s optimised client verifies it 16 $\times$ faster than our Python verifier (87 ms vs. 1.37 s on one thread), which shows how much an optimised implementation can gain.

Table 4: Ratios on matched models ($\uparrow$: ours better, $\downarrow$: ours worse).

System	Model	prover	verifier	proof
zkCNN	LeNet-5	27 $\times\uparrow$	5.0 $\times\downarrow$	1.8 $\times\downarrow$
Bionetta	LeNet-5	230 $\times\uparrow$	2.9 $\times\downarrow$	149 $\times\downarrow$
zkCNN	VGG-16	973 $\times\uparrow$	3.3 $\times\downarrow$	10 $\times\downarrow$
ZKML	VGG-16	7,024 $\times\uparrow$	20 $\times\downarrow$	285 $\times\downarrow$
zkGPT	GPT-2 (64)	50 $\times\uparrow$	2.8 $\times\downarrow$	615 $\times\downarrow$
DeepProve	GPT-2 (64)	78 $\times\uparrow$	1.4 $\times\uparrow$	2.9 $\times\downarrow$
DeepProve	GPT-2 (512)	277 $\times\uparrow$	1.3 $\times\downarrow$	8.4 $\times\downarrow$
zkLLM	OPT-125M (2,048)	49 $\times\uparrow$	65 $\times\downarrow$	5,195 $\times\downarrow$
zkLLM	OPT-6.7B (2,048)	11 $\times\uparrow$	97 $\times\downarrow$	64,338 $\times\downarrow$
zkLLM	Llama-2-7B (2,048)	19 $\times\uparrow$	94 $\times\downarrow$	63,313 $\times\downarrow$
ZKTorch	Llama-2-7B (64)	229 $\times\uparrow$	14 $\times\uparrow$	33 $\times\downarrow$
Maverick	Qwen3-4B (8)	1.4 $\times\downarrow$	16 $\times\downarrow$	same

5 Discussion

What this means. Sampling-based proofs of inference are only as strong as the adversary is weak. They catch a provider who honestly runs the wrong model. They do not catch one who runs the right model and edits the answer, and no sampling rule can close that gap cheaply. Our defence shows that full soundness does not need zkSNARK-level prover cost. Checking each weight layer with a random linear combination costs the provider little more than inference, and gives a $2^{-\lambda}$ guarantee with no trusted setup and no assumption about how models behave. It suits settings where the client has a capable machine and a fast link, for example an enterprise gateway or an auditor who checks a random fraction of queries. It is also most attractive for large models with short prompts, where the proof is much smaller than the model.

Limitations. (i) The proof grows with the activations: megabytes for CNNs, and gigabytes for 7B models at long prompts, where SNARK proofs stay at kilobytes. (ii) Our verifier recomputes attention and other non-weight operations, so its cost grows with the square of the prompt length. (iii) The check is exact, so the provider must serve the integer model. For our CNNs this changed accuracy by at most 0.4 points, but we did not measure the quality of int8 LLMs, whose costs we measured with random weights (and extrapolated beyond 12 blocks). (iv) Like the original protocol, we assume the weight commitment is honest. (v) The protocol is not zero-knowledge: the opened columns reveal a few random linear combinations of each weight row. (vi) Our prototype is Python. Maverick’s client shows that an optimised implementation can be an order of magnitude faster.

Future work. The largest gains would come from (1) compressing the claims, for example with a sum-check argument for the non-weight operations or batching several queries against one set of openings; (2) an optimised verifier (native code, or a GPU); and (3) support for token-by-token generation with a KV cache. Before any wider publication, we intend to share the attack with the authors of [2].

6 Conclusion

We studied a recent lightweight proof of inference, which checks a model’s computation along one random path. We showed that a single tampered neuron defeats it, with 99.8% acceptance and

zero cost on clean inputs, and that choosing the checked nodes more cleverly cannot help. We then built a defence that checks every layer at once with Freivalds' algorithm against a Reed-Solomon/Merkle commitment to the weights. It catches any wrong value with probability $1 - 2^{-\lambda}$. It rejected every one of 2,028 attacks on models from LeNet-5 to Llama-2-13B, and its prover is one to four orders of magnitude faster than zkSNARK provers on the same models. The main takeaway is that checking *everything*, cheaply and algebraically, beats checking a few things exactly.

7 Contributions

[To be completed by the team.] Ofek Lavi: [...]. Erel Brazilay: [...]. Edo Kadosh: [...].

References

- [1] Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkitasubramanian. 2017. Liger: Lightweight Sublinear Arguments Without a Trusted Setup. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2087–2104. doi:10.1145/3133956.3134104
- [2] Pranay Anchuri, Matteo Campanelli, Paul Cesaretti, Rosario Gennaro, Tushar M. Jois, Hasan S. Kayman, and Tugce Ozdemir. 2026. Towards Verifiable AI with Lightweight Cryptographic Proofs of Inference. In *2026 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*. IEEE, 955–977. doi:10.1109/SaTML68715.2026.00058
- [3] Arasu Arun, Adam St. Arnaud, Alexey Titov, Brian Wilcox, Viktor Kolobaric, Marc Brinkmann, Oguzhan Ersoy, Ben Fielding, and Joseph Bonneau. 2025. Verde: Verification via Refereed Delegation for Machine Learning Programs. arXiv:2502.19405.
- [4] Wyatt Benno, Alberto Centelles, Antoine Douchet, and Khalil Gibran. 2026. Jolt Atlas: Verifiable Inference via Lookup Arguments in Zero Knowledge. arXiv:2602.17452.
- [5] Bing-Jyue Chen, Lilia Tang, and Daniel Kang. 2025. ZKTorch: Compiling ML Inference to Zero-Knowledge Proofs via Parallel Proof Accumulation. arXiv:2507.07031.
- [6] Bing-Jyue Chen, Suppakit Waiwitlikhit, Ion Stoica, and Daniel Kang. 2024. ZKML: An Optimizing System for ML Inference in Zero-Knowledge Proofs. In *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys)*. ACM, 560–574. doi:10.1145/3627703.3650088
- [7] KD Conway, Cathie So, Xiaohang Yu, and Kartan Wong. 2024. opML: Optimistic Machine Learning on Blockchain. arXiv:2401.17555.
- [8] Boyuan Feng, Lianke Qin, Zhenfei Zhang, Yufei Ding, and Shumo Chu. 2021. ZEN: An Optimizing Compiler for Verifiable, Zero-Knowledge Neural Network Inferences. Cryptology ePrint Archive, Paper 2021/087. <https://eprint.iacr.org/2021/087>
- [9] Boyuan Feng, Zheng Wang, Yuke Wang, Shu Yang, and Yufei Ding. 2024. ZENO: A Type-based Optimization Framework for Zero Knowledge Neural Network Inference. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 450–464. doi:10.1145/3617232.3624852
- [10] Amos Fiat and Adi Shamir. 1986. How to Prove Yourself: Practical Solutions to Identification and Signature Problems. In *Advances in Cryptology – CRYPTO '86 (LNCS, Vol. 263)*. Springer, 186–194.
- [11] Rusins Freivalds. 1977. Probabilistic Machines Can Use Less Running Time. In *Information Processing 77, Proceedings of the IFIP Congress*. North-Holland, 839–842.
- [12] Nicolas Gailly, Ismael Hishon-Rezaizadeh, Tianyi Liu, Nicholas Mainardi, Dimitrios Papadopoulos, Charalampos Papamanthou, Christodoulos Pappas, Shravan Srinivasan, Zack Youell, and Yupeng Zhang. 2026. DeepProve: Verifiable End-to-End Large Language Model Inference. Cryptology ePrint Archive, Paper 2026/1112. To appear at ACM CCS 2026.
- [13] Zahra Ghodsi, Tianyu Gu, and Siddharth Garg. 2017. SafetyNets: Verifiable Execution of Deep Neural Networks on an Untrusted Cloud. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 30.
- [14] Shafi Goldwasser, Yael Tauman Kalai, and Guy N. Rothblum. 2008. Delegating Computation: Interactive Proofs for Muggles. In *Proceedings of the 40th Annual ACM Symposium on Theory of Computing (STOC)*. ACM, 113–122.
- [15] Tianyu Gu, Kang Liu, Brendan Dolan-Gavitt, and Siddharth Garg. 2019. BadNets: Evaluating Backdoor Attacks on Deep Neural Networks. *IEEE Access* 7 (2019), 47230–47244.
- [16] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. 2018. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2704–2713.
- [17] Daniel Kang, Tatsunori Hashimoto, Ion Stoica, and Yi Sun. 2022. Scaling up Trustless DNN Inference with Zero-Knowledge Proofs. arXiv:2210.08674.
- [18] Kyeongtae Lee, Byeongkyu Han, Jihye Kim, and Hyunok Oh. 2026. LAMP: Linear Verification of Matrix Multiplication via Proximity Testing. Cryptology ePrint Archive, Paper 2026/1571.
- [19] Seunghwa Lee, Hankyung Ko, Jihye Kim, and Hyunok Oh. 2024. vCNN: Verifiable Convolutional Neural Network Based on zk-SNARKs. *IEEE Transactions on Dependable and Secure Computing* 21, 4 (2024), 4254–4270. doi:10.1109/TDSC.2023.3348760
- [20] Youki Lim and Sam Yong. 2026. Seal, Then Sample: Sampled Layerwise Proofs for Verifiable LLM Inference from GPT-2 to 70B. arXiv:2609.27367.
- [21] Tianyi Liu, Xiang Xie, and Yupeng Zhang. 2021. zkCNN: Zero Knowledge Proofs for Convolutional Neural Network Predictions and Accuracy. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2968–2985. doi:10.1145/3460120.3485379
- [22] Ben Merbaum, Mohammad Amin Raesi, Wenhao Wang, Charalampos Papamanthou, Katerina Sotiraki, and Fan Zhang. 2026. Maverick: Private and Verifiable LLM Inference Made Practical via Matrix-Vector Multiplication Delegation. arXiv:2609.10264.
- [23] Ralph C. Merkle. 1987. A Digital Signature Based on a Conventional Encryption Function. In *Advances in Cryptology – CRYPTO '87 (LNCS, Vol. 293)*. Springer, 369–378.
- [24] Jack Min Ong, Matthew Di Ferrante, Aaron Pazdera, Ryan Garner, Sami Jaghouar, Manveer Basra, Max Ryabinin, and Johannes Hagemann. 2025. TOPLOC: A Locality Sensitive Hashing Scheme for Trustless Verifiable Inference. In *Proceedings of the 42nd International Conference on Machine Learning (ICML) (PMLR, Vol. 267)*. 47196–47211.
- [25] Wenjie Qu, Yijun Sun, Xuanming Liu, Tao Lu, Yanpei Guo, Kai Chen, and Jiaheng Zhang. 2025. zkGPT: An Efficient Non-interactive Zero-knowledge Proof Framework for LLM Inference. In *34th USENIX Security Symposium*. USENIX Association, 2045–2063.
- [26] Irving S. Reed and Gustave Solomon. 1960. Polynomial Codes over Certain Finite Fields. *J. Soc. Indust. Appl. Math.* 8, 2 (1960), 300–304.
- [27] Haochen Sun, Jason Li, and Hongyang Zhang. 2024. zkLLM: Zero Knowledge Proofs for Large Language Models. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 4405–4419. doi:10.1145/3658644.3670334
- [28] Yifan Sun, Yuhang Li, Yue Zhang, Yuchen Jin, and Huan Zhang. 2024. SVIP: Towards Verifiable Inference of Open-source Large Language Models. arXiv:2410.22307.
- [29] Florian Tramèr and Dan Boneh. 2019. Slalom: Fast, Verifiable and Private Execution of Neural Networks in Trusted Hardware. In *International Conference on Learning Representations (ICLR)*.
- [30] Zhaohui Wang. 2026. NanoZK: Privacy-Preserving Verifiable Inference for Large Language Models via Layerwise Zero-Knowledge Proofs. arXiv:2603.18046.
- [31] Chenkai Weng, Kang Yang, Xiang Xie, Jonathan Katz, and Xiao Wang. 2021. Mystic: Efficient Conversions for Zero-Knowledge Proofs with Applications to Machine Learning. In *30th USENIX Security Symposium*. USENIX Association, 501–518.
- [32] Tiancheng Xie, Tao Lu, Zhiyong Fang, Siqi Wang, Zhenfei Zhang, Yongzheng Jia, Dawn Song, and Jiaheng Zhang. 2025. zkPyTorch: A Hierarchical Optimized Compiler for Zero-Knowledge Machine Learning. Cryptology ePrint Archive, Paper 2025/535.
- [33] Jianzhu Yao, Hongxu Su, Taobo Liao, Zerui Cheng, Huan Zhang, Xuechao Wang, and Pramod Viswanath. 2026. TAO: Tolerance-Aware Optimistic Verification for Floating-Point Neural Networks. In *Proceedings of the 21st European Conference on Computer Systems (EuroSys)*. ACM, 1515–1532. doi:10.1145/3767295.3803612
- [34] Dmytro Zakharov, Oleksandr Kurbatov, Artem Sdobnov, Lev Soukhanov, Yevhenii Sekhin, Vitalii Volovyk, Mykhailo Velykodnyi, Mark Cherepovskyi, Kyrylo Baibula, Lasha Antadze, Pavlo Kravchenko, Volodymyr Dubinin, and Yaroslav Panasenko. 2025. Bionetta: Efficient Client-Side Zero-Knowledge Machine Learning Proving. arXiv:2510.06784.
- [35] Zkonduit. 2026. EZKL: An Engine for Doing Inference for Deep Learning Models and Other Computational Graphs in a zk-SNARK. <https://github.com/zkonduit/ezkl>. GitHub repository, accessed September 2026.